1 GemTextareaInput

The component is a text area with an optional label. It is used whenever a multi-line user text input is required, e.g. for comment fields, feedback, etc.
Example in 1 demonstrates usage examples of the component.

Listing 1: GemTextareaInputExample

```
package example_input;

import Domain.Person;
import mc.fenix.input.GemTextareaInput;

page GemTextareaInputExample(List<Person>persons) {
  @iterate(Person p:persons) {
    @GemTextareaInput(
      labelText = p.name+":",
      entry = p.aboutMe
    );
  }
}
```

1.1 API Parameters

Table 1 lists the parameters that configure the `GemTextareaInput` component.

Table 1: GemTextareaInput API Parameters

Parameter	Type	Default Value
`id`	`String`	—
Deprecated		
`labelText`	`String`	—
Text describing the input field.		
`entry`	`?String`	—
The input field's value. If it is linked to a variable, the value of the variable will change when the user enters a new value.		
`disabled`	`boolean`	`false`
Whether the input field is disabled.		
`placeholder`	`String`	`""`
Text displayed when the field is empty.		
`width`	`String`	`"100%"`
Width of the input field.		
`height`	`String`	`"100%"`
Height of the input field.		

Continued on next page

Table 1 – *Continued from previous page*

Parameter	Type	Default Value
`validity`	`GemTextareaInputValidity`	—
An object representing the validation status of the field.		
`keyRelease`	`Event<Object>`	—
Callback that is executed when a key is released.		
`keyPress`	`Event<Object>`	—
Callback that is executed when a key is pressed.		
`resize`	`String`	`"auto"`
Option how to let the user resize the InputArea. Possible options: none,both,horizontal,vertical,initial,inherit		

1.2 Related Types and Components

This section outlines both the types and components related to the `GemTextareaInput` module.

1.2.1 `GemTextareaInputTypes` Class Diagram (Listing 2)

Types used in a GemTextareaInput component.

Listing 2: GemTextareaInputTypes

```
package mc.fenix.input;

classdiagram GemTextareaInputTypes {

  public class GemTextareaInputValidity {
    String status;
    String message;
  }

}
```

Class `GemTextareaInputValidity`

An object representing the validation status of the field.

Attributes

Table 2 lists the attributes defined for the `GemTextareaInputValidity` class.

Table 2: Attributes of `GemTextareaInputValidity`

Name	Type	Modifiers
`status`	`String`	—
The validation status. Possible values: • `"Error"`: There is a problem or issue that needs attention. • `"Unchecked"`: The input has not been inspected. • `"Success"`: Positive validation result. • `"Warning"`: There is a potential issue that should be noted.		
`message`	`String`	—
A message to be displayed as a tooltip with the status		

Associations

This class has no associations.

1.2.2 Related Components

Table 3 lists the components utilized by the `GemTextareaInput`, along with their associated references.

Table 3: Components and Their References

Component Name	Reference
`GemText`	15

2 GemDropdownInput

The component is a dropdown field with an optional label. It is used whenever a user input is required, in which the user selects one from multiple options
Example in 3 demonstrates usage examples of the component.

Listing 3: GemDropdownInputExample

```
package example_input;

import mc.fenix.input.GemDropdownInput;
import mc.fenix.input.GemDropdownInputTypes.Option;
import mc.fenix.arrange.GemColumn;
import mc.fenix.arrange.GemRow;

page GemDropdownInputExample() {
  @GemRow(components = [
    @GemDropdownInput(
      width = "50%",
      labelText = "Academic degree:",
      options = [Option("Bachelor","B.Sc."), Option("Master","M.Sc."), Option("PhD")]
    )
  ]);

  @GemColumn(
    rowGap = "15px",
    components = [
      @GemRow(components = [
        @GemDropdownInput(
```

```
        width = "50%",
        labelText = "Hobby:",
        options = [Option("Tennis"), Option("Sleeping"), Option("Model-driven development"),
            Option("Video gaming"), Option("Photographing"), Option("Bouldering"), Option("
            Football"), Option("Drawing"), Option("Social Media"), Option("Knitting"), Option("
            Volleyball"), Option("Running"), Option("Swimming"), Option("Reading"), Option("
            Music"), Option("Movies"), Option("Hiking"), Option("Climbing"), Option("Travelling
            "), Option("Scuba diving"), Option("Hacking"), Option("Cooking"), Option("Gardening
            "), Option("Other")],
        searchEnabled = true
      )
    ]),
    @GemRow(components = [
      @GemDropdownInput(
        width = "50%",
        labelText = "How often do you attend your Hobby?",
        options = [Option("Once a year"), Option("Once a month"), Option("Once a week"), Option
            ("Three times a week"), Option("Everyday"), Option("More than once a day")]
      )
    ])
  ]
 );
}
```

2.1 API Parameters

Table 4 lists the parameters that configure the `GemDropdownInput` component.

Table 4: GemDropdownInput API Parameters

Parameter	Type	Default Value
`id`	`String`	—
Can be set to ensure the list of options is specific to the input field.		
`labelText`	`String`	`""`
Text describing the input field.		
`options`	`List<Option>`	—
A list of options for the dropdown.		
`entry`	`?String`	—
The input field's value. If it is linked to a variable, the value of the variable will change when the user enters a new value.		
`disabled`	`boolean`	`false`
Whether the input field is disabled.		
`width`	`String`	`"100%"`
Width of the input field.		
`height`	`String`	`"100%"`
Height of the input field.		

Continued on next page

Table 4 – *Continued from previous page*

Parameter	Type	Default Value
`validity`	`GemTextInputValidity`	—
An object representing the validation status of the field.		
`searchEnabled`	`boolean`	`false`
Whether a search input to filter the options shows up on top of the list of options		
`keyRelease`	`Event<Object>`	—
Callback that is executed when a key is released.		
`keyPress`	`Event<Object>`	—
Callback that is executed when a key is pressed.		

2.2 Related Types and Components

This section outlines both the types and components related to the `GemDropdownInput` module.

2.2.1 `GemDropdownInputTypes` Class Diagram (Listing 4)

Types used in a GemDropdownInput component.

Listing 4: GemDropdownInputTypes

```
package mc.fenix.input;

classdiagram GemDropdownInputTypes {

  public class Option {
    String value;
    String display;
    +Option(String value);
    +Option(String value, String display);
  }

}
```

Class `Option`

An object representing a dropdown option.

Attributes

Table 5 lists the attributes defined for the `Option` class.

Table 5: Attributes of `Option`

Name	Type	Modifiers
`value`	`String`	—
Value behind the option in a dropdown. An option typically represents an object, e.g. person, via human-readable name. The value is an identifier that represents the object in the logic implementation.		

Continued on next page

Table 5 – *Continued from previous page*

Name	Type	Modifiers
`display`	`String`	—

Human-readable name representing the option.

Associations

This class has no associations.

2.2.2 `GemTextInputTypes` Class Diagram (Listing 5)

Types used in a GemTextInput component.

Listing 5: GemTextInputTypes

```
package mc.fenix.input;

classdiagram GemTextInputTypes {

  public class GemTextInputValidity {
    String status;
    String message;
  }

}
```

Class `GemTextInputValidity`

An object representing the validation status of the field.

Attributes

Table 6 lists the attributes defined for the `GemTextInputValidity` class.

Table 6: Attributes of `GemTextInputValidity`

Name	Type	Modifiers
`status`	`String`	—

The validation status. Possible values:

- `"Error"`: There is a problem or issue that needs attention.
- `"Unchecked"`: The input has not been inspected.
- `"Success"`: Positive validation result.
- `"Warning"`: There is a potential issue that should be noted.

`message`	`String`	—

A message to be displayed as a tooltip with the status

Associations

This class has no associations.

2.2.3 Related Components

Table 7 lists the components utilized by the `GemDropdownInput`, along with their associated references.

Table 7: Components and Their References

Component Name	Reference
`GemTextInput`	4
`GemText`	15

3 GemCheckBoxInput

The component is a checkbox with a label. It is used whenever an input for a binary choice is required, e.g., agreeing to terms and conditions or opting in to a newsletter.
Example in 6 demonstrates usage examples of the component. Figure 1 illustrates the component's UI.

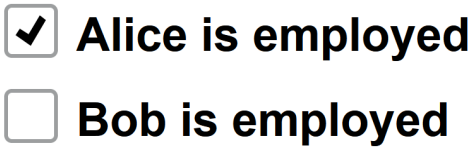
Listing 6: GemCheckBoxInputExample

```
package example_input;

import Domain.Person;
import mc.fenix.input.GemCheckBoxInput;

page GemCheckBoxInputExample(List<Person>persons) {
  @iterate(Person p:persons) {
    @GemCheckBoxInput(
      labelText = p.name+" is employed",
      checked = p.employed
    );
  }
}
```

Figure 1: GUI Visualization for `GemCheckBoxInputExample`.



3.1 API Parameters

Table 8 lists the parameters that configure the `GemCheckBoxInput` component.

Table 8: GemCheckBoxInput API Parameters

Parameter	Type	Default Value
`id`	`String`	—
A unique id ensures proper screen reader functionality for the checkbox description.		
`labelPosition`	`String`	`"right"`
The position of the label, can be `"left"` or `"right"`.		
`labelText`	`String`	`""`
The text of the checkbox label.		

Continued on next page

Table 8 – *Continued from previous page*

Parameter	Type	Default Value
`disabled`	`boolean`	`false`
Whether the input field is disabled.		
`checked`	`?boolean`	—
Indicates if the checkbox is checked (`true`) or not (`false`).		

4 GemTextInput

The component is an input field with an optional label. It is used whenever a user input is required, e.g. to create/update objects, provide a search field, etc.
Example in 7 demonstrates usage examples of the component. Figure 2 illustrates the component's UI.

Listing 7: GemTextInputExample

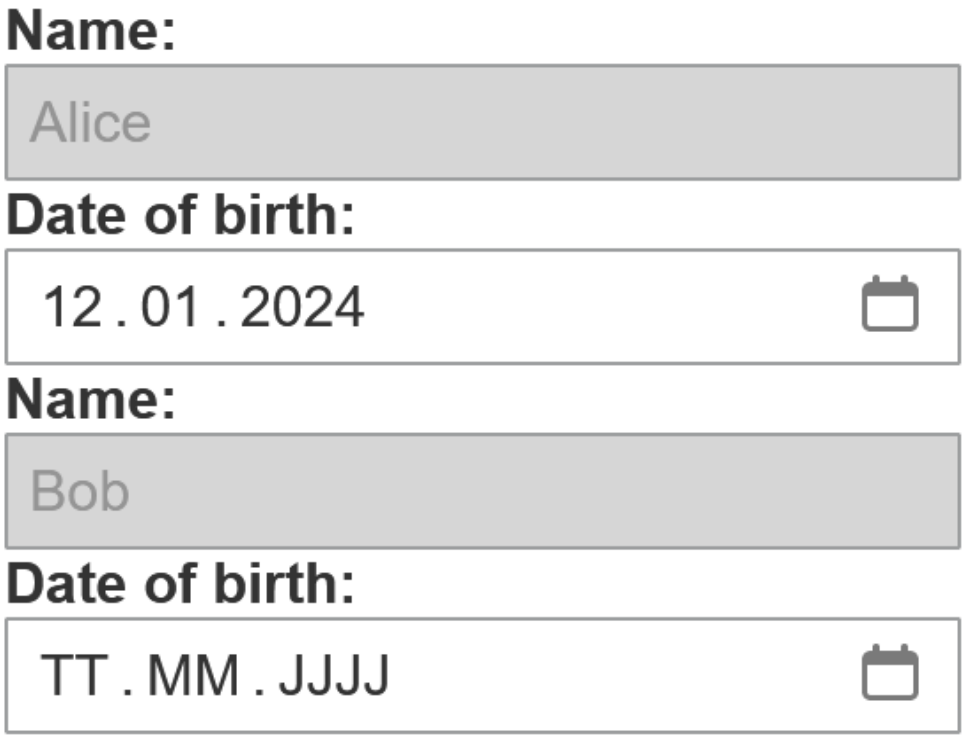
```
package example_input;

import Domain.Person;
import mc.fenix.input.GemTextInput;
import mc.fenix.input.GemDropdownInput;
import mc.fenix.input.GemDropdownInputTypes.Option;

page GemTextInputExample(List<Person>persons) {
  @iterate(Person p:persons) {
    @GemTextInput(
      id = "name",
      labelText = "Name:",
      entry = p.name,
      disabled = true
    );

    @GemTextInput(
      inputType = "date",
      labelText = "Date of birth:",
      entry = p.birthday
    );
  }
}
```

Figure 2: GUI Visualization for `GemTextInputExample`.



4.1 API Parameters

Table 9 lists the parameters that configure the `GemTextInput` component.

Table 9: GemTextInput API Parameters

Parameter	Type	Default Value
`id`	`String`	—
The component's id; used to configure the styles.		
`labelText`	`String`	`""`
Text describing the input field.		
`inputType`	`String`	`"text"`
Type of the input field. The possible types (`https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input#input_types`https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input#input_types) are defined by the browser/platform implementing the input fields. Commonly used types: • `"text"`: Accept any input string. • `"number"`: Accept numbers. • `"date"`: Date input with a calender selection. • `"password"`: Field hiding the input string.		
`entry`	`?String`	—
The input field's value. If it is linked to a variable, the value of the variable will change when the user enters a new value.		

Continued on next page

Table 9 – *Continued from previous page*

Parameter	Type	Default Value
`disabled`	`boolean`	`false`
Whether the input field is disabled.		
`placeholder`	`String`	`""`
Text displayed when the field is empty.		
`width`	`String`	`"100%"`
Width of the input field.		
`height`	`String`	`"100%"`
Height of the input field.		
`validity`	`GemTextInputValidity`	—
An object representing the validation status of the field.		
`keyRelease`	`Event<Object>`	—
Callback that is executed when a key is released.		
`keyPress`	`Event<Object>`	—
Callback that is executed when a key is pressed.		

4.2 Related Types and Components

This section outlines both the types and components related to the `GemTextInput` module.

4.2.1 `GemTextInputTypes` Class Diagram

Further details on the class diagram can be found in 2.2.2.

4.2.2 Related Components

Table 10 lists the components utilized by the `GemTextInput`, along with their associated references.

Table 10: Components and Their References

Component Name	Reference
`GemText`	15

5 GemRadioInput

This component represents a radio button with a label.It is intended for use in forms or other scenarios where users must choose one of several options.
Example in 8 demonstrates usage examples of the component.

Listing 8: GemRadioInputExample

```
1 package example_input;
2
```

```
import mc.fenix.input.GemRadioInput;
import mc.fenix.basic.GemText;

page GemRadioInputExample() {
  @GemText(value = "Group 1");

  @GemRadioInput(
    id = "genderMale",
    labelText = "Male",
    value = "male",
    group = "first"
  );

  @GemRadioInput(
    id = "genderFemale",
    labelText = "Female",
    value = "female",
    group = "first"
  );

  @GemRadioInput(
    id = "genderOther",
    labelText = "Other",
    value = "other",
    group = "first"
  );

  @GemText(value = "Group 2");

  @GemRadioInput(
    id = "a",
    labelText = "A",
    value = "valueA",
    group = "second"
  );

  @GemRadioInput(
    id = "b",
    labelText = "B",
    value = "valueB",
    group = "second"
  );
}
```

5.1 API Parameters

Table 11 lists the parameters that configure the `GemRadioInput` component.

Table 11: GemRadioInput API Parameters

Parameter	Type	Default Value
`id`	`String`	—
`labelText`	`String`	—
`value`	`String`	—
`labelPosition`	`String`	—
`disabled`	`boolean`	`false`

Continued on next page

Table 11 – *Continued from previous page*

Parameter	Type	Default Value
selectedValue	?String	—
group	String	—

6 GemAutocompleteInput

The component is a dropdown field with an optional label. It is used whenever a user input is required, in which the user selects one from multiple options
Example in 9 demonstrates usage examples of the component. Figure 3 illustrates the component's UI.

Listing 9: GemAutocompleteInputExample

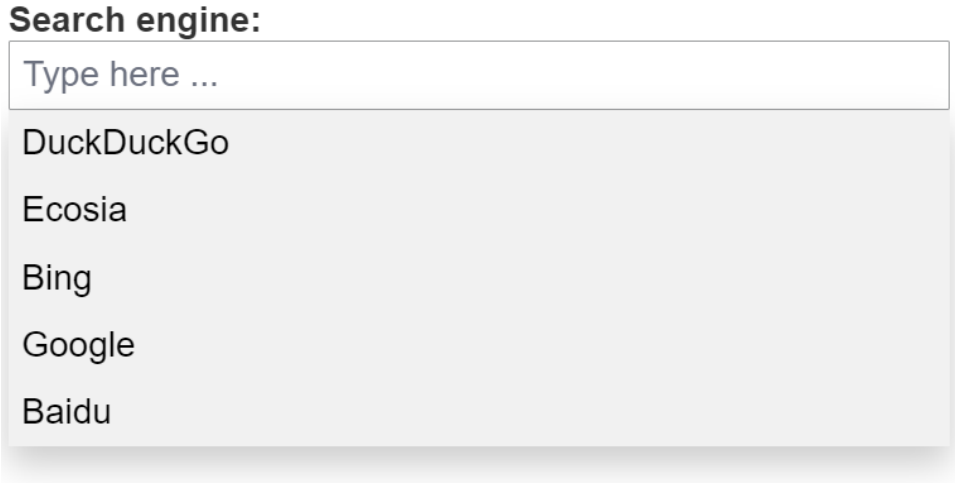
```
package example_input;

import mc.fenix.input.GemAutocompleteInput;
import mc.fenix.input.GemDropdownInputTypes.Option;
import mc.fenix.arrange.GemColumn;
import mc.fenix.arrange.GemRow;

page GemAutocompleteInputExample() {
  @GemRow(components = [
    @GemAutocompleteInput(
      id = "searchEngines",
      width = "50%",
      labelText = "Search engine:",
      placeholder = "Type here ...",
      options = [Option("DuckDuckGo"), Option("Ecosia"), Option("Bing"), Option("Google"),
          Option("Baidu")]
    )
  ]);

  @GemColumn(
    rowGap = "15px",
    components = [
      @GemRow(components = [
        @GemAutocompleteInput(
          width = "50%",
          labelText = "Hobby:",
          options = [Option("Tennis"), Option("Sleeping"), Option("Model-driven development"),
              Option("Video gaming"), Option("Photographing"), Option("Bouldering"), Option("
              Football"), Option("Drawing"), Option("Social Media"), Option("Knitting"), Option("
              Volleyball"), Option("Running"), Option("Swimming"), Option("Reading"), Option("
              Music"), Option("Movies"), Option("Hiking"), Option("Climbing"), Option("Travelling
              "), Option("Scuba diving"), Option("Hacking"), Option("Cooking"), Option("Gardening
              "), Option("Other")]
        )
      ])
    ]
  );
}
```

Figure 3: GUI Visualization for `GemAutocompleteInputExample`.



6.1 API Parameters

Table 12 lists the parameters that configure the `GemAutocompleteInput` component.

Table 12: GemAutocompleteInput API Parameters

Parameter	Type	Default Value
`id`	`String`	—
Can be set to ensure the list of options is specific to the input field.		
`labelText`	`String`	`""`
Text describing the input field.		
`options`	`List<Option>`	—
A list of options for the autocomplete suggestions		
`entry`	`?String`	—
The input field's value. If it is linked to a variable, the value of the variable will change when the user enters a new value.		
`disabled`	`boolean`	`false`
Whether the input field is disabled.		
`width`	`String`	`"100%"`
Width of the input field.		
`height`	`String`	`"100%"`
Height of the input field.		
`validity`	`GemTextInputValidity`	—
An object representing the validation status of the field.		

Continued on next page

Table 12 – *Continued from previous page*

Parameter	Type	Default Value
`placeholder`	`String`	`""`
Text displayed when the field is empty.		
`keyRelease`	`Event<Object>`	—
Callback that is executed when a key is released.		
`keyPress`	`Event<Object>`	—
Callback that is executed when a key is pressed.		

6.2 Related Types and Components

This section outlines both the types and components related to the `GemAutocompleteInput` module.

6.2.1 `GemDropdownInputTypes` Class Diagram

Further details on the class diagram can be found in 2.2.1.

6.2.2 `GemTextInputTypes` Class Diagram

Further details on the class diagram can be found in 2.2.2.

6.2.3 Related Components

Table 13 lists the components utilized by the `GemAutocompleteInput`, along with their associated references.

Table 13: Components and Their References

Component Name	Reference
`GemDropdownInput`	2
`GemTextInput`	4
`GemText`	15

7 GemDualRangeInput

7.1 API Parameters

Table 14 lists the parameters that configure the `GemDualRangeInput` component.

Table 14: GemDualRangeInput API Parameters

Parameter	Type	Default Value
`id`	`String`	—
`labelText`	`String`	—
`sliderBehavior`	`String`	—
`vertical`	`Boolean`	—

Continued on next page

Table 14 – *Continued from previous page*

Parameter	Type	Default Value
step	Integer	—
minValue	Integer	—
maxValue	Integer	—
startValue	?Integer	—
endValue	?Integer	—

8 GemRangeInput

This component displays a label with a slider that allows users to select a value within a specified range.This component can be used in forms and other interactive interfaces to allow users to select a range of values easily.
Example in 10 demonstrates usage examples of the component.

Listing 10: GemRangeInputExample

```
package example_input;

import mc.fenix.input.GemRangeInput;

page GemRangeInputExample() {
  @GemRangeInput(
    id = "rangeInput5",
    labelText = "Range1: ",
    vertical = false,
    step = 1,
    minValue = 1,
    maxValue = 100
  );
}
```

8.1 API Parameters

Table 15 lists the parameters that configure the `GemRangeInput` component.

Table 15: GemRangeInput API Parameters

Parameter	Type	Default Value
id	String	—
labelText	String	—
vertical	Boolean	—
step	Integer	—
minValue	Integer	—
maxValue	Integer	—
value	?Integer	—

9 GemImage

The image component embeds image on a page.
Example in 11 demonstrates usage examples of the component.

Listing 11: GemImageExample

```
package example_basic;

import mc.fenix.basic.GemImage;
import mc.fenix.arrange.GemCard;

page GemImageExample() {
  @GemCard(component = @GemImage(
    source = "assets/images/World.png",
    alt = "Image of the Earth"
  ));

  @GemImage(
    width = "300px",
    source = "assets/images/World.png",
    alt = "Image of the Earth"
  );

  @GemImage(
    height = "300px",
    source = "assets/images/World.png",
    alt = "Image of the Earth"
  );

  @GemImage(
    width = "30px",
    height = "30px",
    source = "assets/images/World.png",
    alt = "Image of the Earth"
  );
}
```

9.1 API Parameters

Table 16 lists the parameters that configure the `GemImage` component.

Table 16: GemImage API Parameters

Parameter	Type	Default Value
`height`	`String`	`"auto"`
Height of the component in `px`, `%`, `vh`. By default takes up all available space while preserving aspect ratio.		
`width`	`String`	`"auto"`
Width of the component in `px`, `%`, `vh`. By default takes up all available space while preserving aspect ratio.		
`source`	`String`	—
The path to the image file. It can be a URL or a relative path. The relative path starts from the root folder (typically `src/`).		

Continued on next page

Table 16 – *Continued from previous page*

Parameter	Type	Default Value
`alt`	`String`	`""`
The text that is shown if image could not be loaded.		
`style`	`String`	`""`
Deprecated.		

10 GemIcon

Displays an icon on a webpage.
Example in 12 demonstrates usage examples of the component.

Listing 12: GemIconExample

```
package example_basic;

import mc.fenix.basic.GemIcon;
import mc.fenix.basic.GemButton;
import mc.fenix.arrange.GemRow;

page GemIconExample() {
  @GemRow(
    colGap = "15px",
    components = [
      @GemIcon(
        name = "add",
        color = "blue",
        size = "1rem"
      ),
      @GemIcon(
        name = "edit",
        color = "rgb(95,99,104)",
        size = "24px"
      ),
      @GemIcon(
        name = "delete",
        color = "#ff0000",
        size = "2cm"
      )
    ]
  );

  @GemButton(
    width = "fit-content",
    content = [
      @GemIcon(
        name = "home",
        color = "white",
        size = "30px"
      )
    ]
  );
}
```

10.1 API Parameters

Table 17 lists the parameters that configure the `GemIcon` component.

Table 17: GemIcon API Parameters

Parameter	Type	Default Value
name	String	—
color	String	—
size	String	—

11 GemLink

The link element is used to create a hyperlink to another web page or to a specific section within a page.
Example in 13 demonstrates usage examples of the component.

Listing 13: GemLinkExample

```
package example_basic;

import mc.fenix.basic.GemLink;

page GemLinkExample() {
  @GemLink(
    url = "https://www.google.com/",
    text = "click here for Google"
  );
}
```

11.1 API Parameters

Table 18 lists the parameters that configure the `GemLink` component.

Table 18: GemLink API Parameters

Parameter	Type	Default Value
url	String	""
URL the hyperlink redirects to.		
text	String	""
Text of the link.		

12 GemVideo

The component shows a video on a web page.
Example in 14 demonstrates usage examples of the component.

Listing 14: GemVideoExample

```
package example_basic;

import mc.fenix.basic.GemVideo;

page GemVideoExample() {
  @GemVideo(source = "https://archive.org/download/BigBuckBunny_124/Content/
      big_buck_bunny_720p_surround.mp4");
}
```

12.1 API Parameters

Table 19 lists the parameters that configure the `GemVideo` component.

Table 19: GemVideo API Parameters

Parameter	Type	Default Value
`source`	`String`	—
The path to the video file. It can be a URL or a relative path. The relative path starts from the root folder (typically `src/`).		

13 GemButton

Button enables user interaction through simple clicks or taps, triggering actions or tasks.
Example in 15 demonstrates usage examples of the component. Figure 4 illustrates the component's UI.

Listing 15: GemButtonExample

```
package example_basic;

import mc.fenix.basic.GemButton;

page GemButtonExample() {
  @GemButton(
    label = "windows alert",
    leftClick = alert()
  );

  @GemButton(
    label = "Cannot click me",
    disabled = true
  );
}
```

Figure 4: GUI Visualization for `GemButtonExample`.



13.1 API Parameters

Table 20 lists the parameters that configure the `GemButton` component.

Table 20: GemButton API Parameters

Parameter	Type	Default Value
`height`	`String`	`"auto"`
Height of the component in `px`, `%`, `vh`. By default collapses to a minimal possible height or takes up all available space depending on context.		

Continued on next page

Table 20 – *Continued from previous page*

Parameter	Type	Default Value
`width`	`String`	`"auto"`
Width of the component in `px`, `%`, `vh`. By default collapses to a minimal possible width or takes up all available space depending on context.		
`label`	`String`	`""`
Button text.		
`content`	`List<GUIViewElement>`	—
Button content if a more complex element than text is required. If label and content are both set, only content is shown.		
`disabled`	`boolean`	`false`
If true, the button cannot be clicked.		
`leftClick`	`Event<Void>`	—
Callback that is executed when a button is clicked.		
`routerLink`	`String`	`null`
`routerLinkActive`	`String`	`null`

14 GemAudio

The audio component embeds sound content on a page.
Example in 16 demonstrates usage examples of the component.

Listing 16: GemAudioExample

```
package example_basic;

import mc.fenix.basic.GemAudio;

page GemAudioExample() {
  @GemAudio(
    width = "300px",
    source = "assets/audio/audio.mp3"
  );
}
```

14.1 API Parameters

Table 21 lists the parameters that configure the `GemAudio` component.

Table 21: GemAudio API Parameters

Parameter	Type	Default Value
`width`	`String`	`"auto"`
Width of the component in `px`, `%`, `vh`. By default takes up all available space.		

Continued on next page

Table 21 – *Continued from previous page*

Parameter	Type	Default Value
source	String	—
The path to the audio file. It can be a URL or a relative path. The relative path starts from the root folder (typically src/).		

15 GemText

Displays text on a webpage.
Example in 17 demonstrates usage examples of the component.

Listing 17: GemTextExample

```
package example_basic;

import mc.fenix.basic.GemText;

page GemTextExample() {
  @GemText(
    value = "My text",
    color = "blue",
    size = "1rem",
    fontFamily = "'Lucida Console', Courier, monospace",
    fontStyle = "oblique 12deg",
    weight = "200",
    width = "200px",
    height = "3rem"
  );
}
```

15.1 API Parameters

Table 22 lists the parameters that configure the GemText component.

Table 22: GemText API Parameters

Parameter	Type	Default Value
value	String	—
Text content.		
color	String	—
size	String	—
fontStyle	String	"unset"
Font style. Can be "normal", "italic", "oblique <deg>", or have a universal value		
weight	String	"unset"
Font weight. Can be "normal", "bold", "lighter", "<number>", or have a universal value		
fontFamily	String	"unset"
Font family. Example: "Gill Sans", sans-serif		

Continued on next page

Table 22 – *Continued from previous page*

Parameter	Type	Default Value
`width`	`String`	`"100%"`
Width of the input field.		
`height`	`String`	`"100%"`
Height of the input field.		

16 GemCheckbox

This component represents a checkbox with a label. It is designed to be used in forms or other contexts where the user needs to indicate a binary choice,such as agreeing to terms and conditions or opting in to a newsletter.A single instance of the component results in a single checkbox input.

16.1 API Parameters

Table 23 lists the parameters that configure the `GemCheckbox` component.

Table 23: GemCheckbox API Parameters

Parameter	Type	Default Value
`checkboxItem`	`String`	—

17 GemTableHeaderCell

GemTableHeaderCell controls dimensions that content should occupy within a GemTable's header.

17.1 API Parameters

Table 24 lists the parameters that configure the `GemTableHeaderCell` component.

Table 24: GemTableHeaderCell API Parameters

Parameter	Type	Default Value
`colSpan`	`int`	`1`
Number of columns a cell should cover		
`rowSpan`	`int`	`1`
Number of rows a cell should cover		
`content`	`GUIViewElement`	—

18 GemTableRowCell

GemTableRowCell controls dimensions that content should occupy within a GemTable's body.

18.1 API Parameters

Table 25 lists the parameters that configure the `GemTableRowCell` component.

Table 25: GemTableRowCell API Parameters

Parameter	Type	Default Value
colSpan	int	1
Number of columns a cell should cover		
rowSpan	int	1
Number of rows a cell should cover		
content	GUIViewElement	—

19 GemTable

A table organizes and displays data in rows and columns.
Example in 18 demonstrates usage examples of the component.

Listing 18: GemTableExample

```
package example_table;

import mc.fenix.table.GemTable;
import mc.fenix.table.GemTableHeader;
import mc.fenix.table.GemTableHeaderCell;
import mc.fenix.table.GemTableRow;
import mc.fenix.table.GemTableRowCell;
import mc.fenix.basic.GemText;
import mc.fenix.input.GemTextInput;
import Domain.*;
import mc.fenix.arrange.GemColumn;

page GemTableExample(List<Person>persons) {
  @GemColumn(
    rowGap = "20px",
    width = "50%",
    components = [
      @GemTable(
        headers = [
          @GemTableHeader(cols = ["Name", "Birthday"])
        ],
        body = [
          @iterate(Person p:persons) {
            @GemTableRow(cols = [
              @GemText(value = p.name),
              @GemTextInput(
                inputType = "date",
                entry = p.birthday
              )
            ]);
          }

        ]
      ),
      @GemTable(
        layout = "fixed",
        headers = [
          @GemTableHeader(cells = [
            @GemTableHeaderCell(
              rowSpan = 2,
              content = @GemText(value = "Name")
```

```
          ),
          @GemTableHeaderCell(
            colSpan = 2,
            content = @GemText(value = "Address")
          )
        ]),
        @GemTableHeader(cols = ["Street", "House"])
      ],
      body = [
        @GemTableRow(cells = [
          @GemTableRowCell(content = @GemText(value = "Emma")),
          @GemTableRowCell(content = @GemText(value = "Oak Str.")),
          @GemTableRowCell(content = @GemText(value = "29"))
        ]),
        @GemTableRow(cells = [
          @GemTableRowCell(content = @GemText(value = "Alice")),
          @GemTableRowCell(
            rowSpan = 2,
            content = @GemText(value = "Gitschiner Str.")
          ),
          @GemTableRowCell(
            rowSpan = 2,
            content = @GemText(value = "9")
          )
        ]),
        @GemTableRow(cells = [
          @GemTableRowCell(content = @GemText(value = "Olivia"))
        ]),
        @GemTableRow(cells = [
          @GemTableRowCell(content = @GemText(value = "Isabella")),
          @GemTableRowCell(content = @GemText(value = "Pine")),
          @GemTableRowCell(content = @GemText(value = "Rd."))
        ])
      ]
    )
  ]
);
}
```

19.1 API Parameters

Table 26 lists the parameters that configure the `GemTable` component.

Table 26: GemTable API Parameters

Parameter	Type	Default Value
`width`	`String`	`"auto"`
Width of the component in `px`, `%`, `vh`. By default collapses to a minimal possible height with respect to content.		
`height`	`String`	`"auto"`
Height of the component in `px`, `%`, `vh`. By default collapses to a minimal possible height with respect to content.		

Continued on next page

Table 26 – *Continued from previous page*

Parameter	Type	Default Value
`layout`	`String`	`"auto"`
Table layout. Commonly used layouts: • `"auto"`: The width of the columns is adjusted to fit its content (default). • `"fixed"`: The width of the columns is evenly spread.		
`headers`	`List<GUIViewElement>`	—
Header(s) of a table. Normally set to a list of `GemTableHeader`.		
`body`	`List<GUIViewElement>`	—
Body of a table. Normally set to a list of `GemTableRow`.		

20 GemTableRow

A table row that is part of the body of the table.

20.1 API Parameters

Table 27 lists the parameters that configure the `GemTableRow` component.

Table 27: GemTableRow API Parameters

Parameter	Type	Default Value
`cols`	`List<GUIViewElement>`	—
Columns of a table row.		
`cells`	`List<GUIViewElement>`	—
Cells of a table row. Normally set to a list of `GemTableRowCell` component that can specify if a cell spans over multiple columns or rows. If `cells` and `cols` are defined, `cells` are used.		

21 GemTableHeader

A table row that identifies the head of the table, adds some default styling to text content.

21.1 API Parameters

Table 28 lists the parameters that configure the `GemTableHeader` component.

Table 28: GemTableHeader API Parameters

Parameter	Type	Default Value
`cols`	`List<String>`	—
Columns of a table header row.		

Continued on next page

Table 28 – *Continued from previous page*

Parameter	Type	Default Value
cells	List<GUIViewElement>	—
Cells of a table header. Normally set to a list of GemTableHeaderCell component that can specify if a cell spans over multiple columns or rows. If cells and cols are defined, cells are used.		

22 GemDialog

Example in 19 demonstrates usage examples of the component.

Listing 19: GemDialogExample

```
package example_arrange;

import mc.fenix.arrange.GemCard;
import mc.fenix.arrange.GemDialog;
import mc.fenix.basic.GemText;
import mc.fenix.basic.GemButton;
import mc.fenix.basic.GemLink;

page GemDialogExample(?boolean dialogOpen = false) {
  @GemButton(
    label = "Click Me!",
    leftClick = openDialog()
  );

  dialog@GemDialog(
    isOpen = dialogOpen,
    component = @GemCard(
      title = "Info Text",
      component = @GemText(value = "Some Text")
    )
  );
}
```

22.1 API Parameters

Table 29 lists the parameters that configure the GemDialog component.

Table 29: GemDialog API Parameters

Parameter	Type	Default Value
enableBackdrop	boolean	true
top	String	"auto"
left	String	"auto"
isOpen	?boolean	false
component	GUIViewElement	—

23 GemGrid

GemGrid is a container which arranges input elements in a grid pattern.
Example in 20 demonstrates usage examples of the component. Figure 5 illustrates the component's UI.

Listing 20: GemGridExample

```
package example_arrange;

import mc.fenix.arrange.GemGrid;
import mc.fenix.arrange.GemGridCell;
import mc.fenix.basic.GemText;

page GemGridExample() {
  @GemGrid(rows = [
    [
      @GemText(value = "1, 1"),
      @GemText(value = "1, 2")
    ],
    [
      @GemText(value = "2, 1"),
      @GemText(value = "2, 2"),
      @GemText(value = "2, 3")
    ]
  ]);

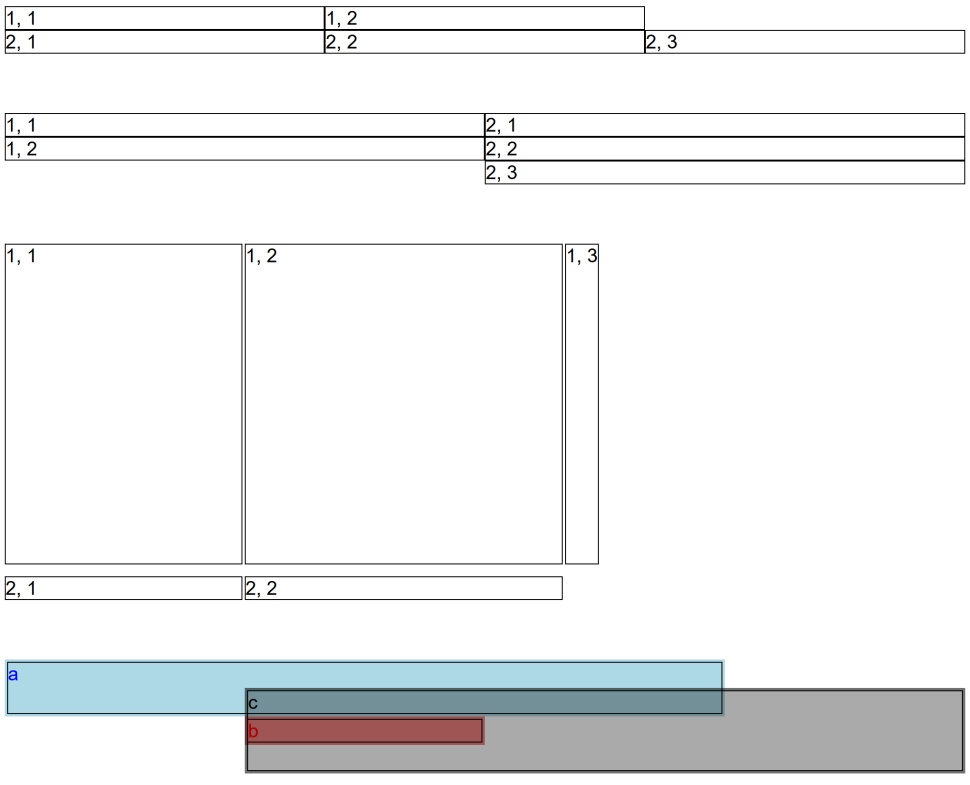
  @GemGrid(cols = [
    [
      @GemText(value = "1, 1"),
      @GemText(value = "1, 2")
    ],
    [
      @GemText(value = "2, 1"),
      @GemText(value = "2, 2"),
      @GemText(value = "2, 3")
    ]
  ]);

  fixedGrid@GemGrid(
    width = "500px",
    height = "300px",
    x = ["200px", "1fr", "auto"],
    y = ["auto", "20px"],
    rowGap = "10px",
    colGap = "2px",
    rows = [
      [
        @GemText(value = "1, 1"),
        @GemText(value = "1, 2"),
        @GemText(value = "1, 3")
      ],
      [
        @GemText(value = "2, 1"),
        @GemText(value = "2, 2")
      ]
    ]
  );

  spanningGrid@GemGrid(
    x = ["1fr", "1fr", "1fr", "1fr"],
    y = ["1fr", "1fr", "1fr", "1fr"],
    cells = [
      cell1@GemGridCell(
        a = [1, 1],
        b = [3, 2],
        content = @GemText(
          width = "100%",
          height = "100%",
          value = "a",
```

```
          color = "blue"
        )
      ),
      cell2@GemGridCell(
        a = [2, 3],
        content = @GemText(
          width = "100%",
          height = "100%",
          value = "b",
          color = "red"
        )
      ),
      cell3@GemGridCell(
        a = [2, 2],
        b = [4, 4],
        content = @GemText(
          width = "100%",
          height = "100%",
          value = "c",
          color = "black"
        )
      )
    ]
  );
}
```

Figure 5: GUI Visualization for `GemGridExample`.



23.1 API Parameters

Table 30 lists the parameters that configure the `GemGrid` component.

Table 30: GemGrid API Parameters

Parameter	Type	Default Value
`height`	`String`	`"auto"`
Height of the component in `px`, `%`, `vh`. By default collapses to a minimal possible height with respect to content.		
`width`	`String`	`"auto"`
Width of the component in `px`, `%`, `vh`. By default collapses to a minimal possible height with respect to content.		
`hAlign`	`String`	`"stretch"`
Horizontal alignment of the embedded components. Invalid values are ignored. Possible values: `"left"`, `"center"`, `"right"`: Set position of the elements inside the container. • `"stretch"`: Stretch elements proportionally to fill space if the elements' size is not specified. • `"space-between"`: Distribute empty space equally between the elements. • `"space-around"`: Distribute empty space around each element equally. • `"space-evenly"`: Distribute empty space around the elements and make those gaps equal size.		
`vAlign`	`String`	`"stretch"`
Vertical alignment of the embedded components. Invalid values are ignored. Possible values: `"top"`, `"center"`, `"bottom"`: Set position of the elements inside the container. • `"stretch"`: Stretch elements proportionally to fill space if the elements' size is not specified. • `"space-between"`: Distribute empty space equally between the elements. • `"space-around"`: Distribute empty space around the elements equally. • `"space-evenly"`: Distribute empty space around the elements and make those gaps equal size.		
`rowGap`	`String`	`"0"`
Gap between rows of elements in the container in `px`, `%`, `vh`, etc.		
`colGap`	`String`	`"0"`
Gap between columns of elements in the container in `px`, `%`, `vh`, etc.		
`x`	`List<String>`	—
Specifies widths of grid columns. The amount of columns is equal to the length of the list.		
`y`	`List<String>`	—
Specifies heights of grid rows. The amount of rows is equal to the length of the list.		
`rows`	`List<List<GUIViewElement>>`	—
Rows of a grid. The length of the longest row defines the amount of columns, unless `x` is specified. If `rows` and `cols` are defined, `rows` are used.		

Continued on next page

Table 30 – *Continued from previous page*

Parameter	Type	Default Value
`cols`	`List<List<GUIViewElement>>`	—
Columns of a grid. The length of the longest column defines the amount of rows, unless `y` is specified.		
`cells`	`List<GUIViewElement>`	—
Cells of a grid. Normally set to a list of `GemGridCell` component that can specify the space occupied by a cell. The cells may overlap. If `cells`, `rows` and `cols` are defined, `cells` are used.		

24 GemOverlayItem

24.1 API Parameters

Table 31 lists the parameters that configure the `GemOverlayItem` component.

Table 31: GemOverlayItem API Parameters

Parameter	Type	Default Value
`component`	`GUIViewElement`	—
`x`	`String`	—
`y`	`String`	—

25 GemCard

Card is a container with a header and a body. Both header and body may contain any elements, such as button, inputs, charts, etc.
Example in 21 demonstrates usage examples of the component. Figure 6 illustrates the component's UI.

Listing 21: GemCardExample

```
package example_arrange;

import mc.fenix.arrange.GemCard;
import mc.fenix.basic.GemText;
import mc.fenix.basic.GemButton;
import mc.fenix.basic.GemLink;

page GemCardExample() {
  @GemCard(
    title = "Info Text",
    component = @GemText(value = "Some Text")
  );

  @GemCard(
    width = "50%",
    height = "300px",
    titleComponent = @GemLink(
      url = "https://www.google.com/",
      text = "Google link"
    ),
    component = @GemButton(label = "A button")
  );
}
```

Figure 6: GUI Visualization for `GemCardExample`.



25.1 API Parameters

Table 32 lists the parameters that configure the `GemCard` component.

Table 32: GemCard API Parameters

Parameter	Type	Default Value
`height`	`String`	`"auto"`
Height of the component in `px`, `%`, `vh`. By default collapses to a minimal possible height with respect to content.		
`width`	`String`	`"auto"`
Width of the component in `px`, `%`, `vh`. By default collapses to a minimal possible width with respect to content.		
`title`	`String`	`""`
Text displayed at the top of the card.		
`titleComponent`	`GUIViewElement`	—
A component displayed at the top of the card. If `titleComponent` and `title` are both set, the `titleComponent` will be used.		
`component`	`GUIViewElement`	—
Component displayed in body of the card.		

25.2 Related Components

Table 33 lists the components utilized by the `GemCard`, along with their associated references.

Table 33: Components and Their References

Component Name	Reference
GemRow	26
GemColumn	27

26 GemRow

GemRow is a container which arranges input elements horizontally.
Example in 22 demonstrates usage examples of the component. Figure 7 illustrates the component's UI.

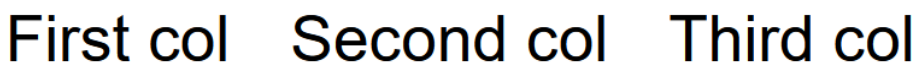
Listing 22: GemRowExample

```
package example_arrange;

import mc.fenix.arrange.GemRow;
import mc.fenix.basic.GemText;

page GemRowExample() {
  @GemRow(
    colGap = "15px",
    components = [
      @GemText(value = "First col"),
      @GemText(value = "Second col"),
      @GemText(value = "Third col")
    ]
  );
}
```

Figure 7: GUI Visualization for GemRowExample.



26.1 API Parameters

Table 34 lists the parameters that configure the GemRow component.

Table 34: GemRow API Parameters

Parameter	Type	Default Value
height	String	"100%"
Height of the component in px, %, vh, etc.		
width	String	"100%"
Width of the component in px, %, vh, etc.		

Continued on next page

Table 34 – *Continued from previous page*

Parameter	Type	Default Value
`hAlign`	`String`	`"stretch"`

Horizontal alignment of the embedded components. Invalid values are ignored. Possible values: `"left"`, `"center"`, `"right"`: Set position of the elements inside the container.

- `"stretch"`: Stretch elements proportionally to fill space if the elements' size is not specified.
- `"space-between"`: Distribute empty space equally between the elements.
- `"space-around"`: Distribute empty space around each element equally.
- `"space-evenly"`: Distribute empty space around the elements and make those gaps equal size.

`vAlign`	`String`	`"stretch"`

Vertical alignment of the embedded components. Invalid values are ignored. Possible values: `"top"`, `"center"`, `"bottom"`: Set position of the elements inside the container.

- `"stretch"`: Stretch elements proportionally to fill space if the elements' size is not specified.
- `"space-between"`: Distribute empty space equally between the elements.
- `"space-around"`: Distribute empty space around the elements equally.
- `"space-evenly"`: Distribute empty space around the elements and make those gaps equal size.

`rowGap`	`String`	`"0"`

Gap between rows of elements in the container in `px`, `%`, `vh`, etc. Relevant when `wrap` is set to `"wrap"` or `"wrap-reverse"`.

`colGap`	`String`	`"0"`

Gap between columns of elements in the container in `px`, `%`, `vh`, etc.

`wrap`	`String`	`"nowrap"`

Set behavior for element wrapping. Invalid values are ignored.

- `"nowrap"`: Elements are never wrapped. If elements do not into the container, a scrollbar appears.
- `"wrap"`: Elements that do not fit into the container are wrapped, i.e. a new column is created to the right.
- `"wrap-reverse"`: Similar to `"wrap"`, but a new column is created to the left.

`components`	`List<GUIViewElement>`	—

Components to display inside a column.

`component`	`GUIViewElement`	—

Deprecated. Shortcut for `components` property for the case when only one component has to be displayed. If both `component` and `components` are defined, `component` is shown first.

`style`	`String`	`""`

Deprecated.

27 GemColumn

GemColumn is a container which arranges input elements vertically.
Example in 23 demonstrates usage examples of the component. Figure 8 illustrates the component's UI.

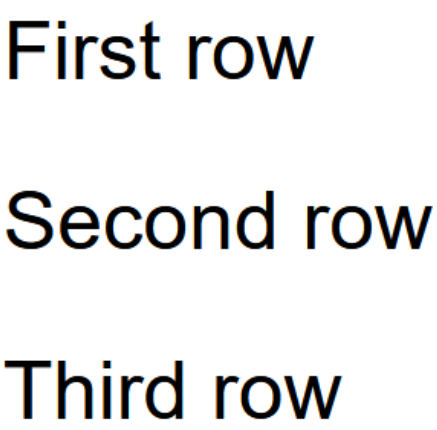
Listing 23: GemColumnExample

```
package example_arrange;

import mc.fenix.arrange.GemColumn;
import mc.fenix.basic.GemText;

page GemColumnExample() {
  @GemColumn(
    rowGap = "15px",
    components = [
      @GemText(value = "First row"),
      @GemText(value = "Second row"),
      @GemText(value = "Third row")
    ]
  );
}
```

Figure 8: GUI Visualization for `GemColumnExample`.



27.1 API Parameters

Table 35 lists the parameters that configure the `GemColumn` component.

Table 35: GemColumn API Parameters

Parameter	Type	Default Value
`height`	`String`	`"100%"`
Height of the component in `px`, `%`, `vh`.		
`width`	`String`	`"100%"`
Width of the component in `px`, `%`, `vh`.		

Continued on next page

Table 35 – *Continued from previous page*

Parameter	Type	Default Value
`hAlign`	`String`	`"stretch"`

Horizontal alignment of the embedded components. Invalid values are ignored. Possible values: `"left"`, `"center"`, `"right"`: Set position of the elements inside the container.

- `"stretch"`: Stretch elements proportionally to fill space if the elements' size is not specified.
- `"space-between"`: Distribute empty space equally between the elements.
- `"space-around"`: Distribute empty space around each element equally.
- `"space-evenly"`: Distribute empty space around the elements and make those gaps equal size.

`vAlign`	`String`	`"stretch"`

Vertical alignment of the embedded components. Invalid values are ignored. Possible values: `"top"`, `"center"`, `"bottom"`: Set position of the elements inside the container.

- `"stretch"`: Stretch elements proportionally to fill space if the elements' size is not specified.
- `"space-between"`: Distribute empty space equally between the elements.
- `"space-around"`: Distribute empty space around the elements equally.
- `"space-evenly"`: Distribute empty space around the elements and make those gaps equal size.

`rowGap`	`String`	`"0"`

Gap between rows of elements in the container in `px`, `%`, `vh`, etc.

`colGap`	`String`	`"0"`

Gap between columns of elements in the container in `px`, `%`, `vh`, etc. Relevant when `wrap` is set to `"wrap"` or `"wrap-reverse"`.

`wrap`	`String`	`"nowrap"`

Set behavior for element wrapping. Invalid values are ignored.

- `"nowrap"`: Elements are never wrapped. If elements do not into the container, a scrollbar appears.
- `"wrap"`: Elements that do not fit into the container are wrapped, i.e. a new column is created to the right.
- `"wrap-reverse"`: Similar to `"wrap"`, but a new column is created to the left.

`components`	`List<GUIViewElement>`	—

Components to display inside a column.

`component`	`GUIViewElement`	—

Deprecated. Shortcut for `components` property for the case when only one component has to be displayed. If both `component` and `components` are defined, `component` is shown first.

`style`	`String`	`""`

Deprecated.

28 GemTreeNode

Combine multiple tree nodes to a tree. A tree displays hierarchical data. It provides a visual representation of parent-child relationships, allowing users to navigate and interact with a hierarchical dataset. Example in 24 demonstrates usage examples of the component.

Listing 24: GemTreeNodeExample

```
package example_arrange;

import Domain.Person;
import mc.fenix.arrange.GemTreeNode;
import mc.fenix.basic.GemText;
import mc.fenix.input.GemCheckBoxInput;

page GemTreeNodeExample(List<Person>persons) {
  @GemTreeNode(children = [
    @iterate(Person p:persons) {
      @GemTreeNode(
        content = @GemText(value = p.name),
        children = [
          @iterate(Person f:p.friends) {
            @GemTreeNode(
              content = @GemText(value = f.name),
              children = [
                @iterate(Person ff:f.friends) {
                  @GemCheckBoxInput(
                    labelText = ff.name+" is employed",
                    checked = ff.employed
                  );
                }

              ]
            );
          }

        ],
        onClick = sayHello(p.name)
      );
    }

  ]);
}
```

28.1 API Parameters

Table 36 lists the parameters that configure the `GemTreeNode` component.

Table 36: GemTreeNode API Parameters

Parameter	Type	Default Value
`isExpanded`	`boolean`	`false`
Whether subtree of this node is expanded		
`content`	`GUIViewElement`	—
Content of this node. If not set, the node is expected to be the root of the tree.		

Continued on next page

Table 36 – *Continued from previous page*

Parameter	Type	Default Value
`children`	`List<GUIViewElement>`	—
Tree nodes. Each node defines a root of a subtree and its children. Normally set to a list of `GemTreeNode`. If not set or empty, the node is expected to be a leaf of the tree.		
`onClick`	`Event<Void>`	—
Callback that is executed when the node is clicked.		
`onToggle`	`Event<boolean>`	—
Callback that is executed when the expand/collapse icon is clicked. Returns `true` if the subtree is expanded, otherwise `false`.		

29 GemOverlay

29.1 API Parameters

Table 37 lists the parameters that configure the `GemOverlay` component.

Table 37: GemOverlay API Parameters

Parameter	Type	Default Value
`background`	`GUIViewElement`	—
`components`	`List<GUIViewElement>`	—

30 GemGridCell

GemGridCell controls dimensions that content should occupy within a GemGrid

30.1 API Parameters

Table 38 lists the parameters that configure the `GemGridCell` component.

Table 38: GemGridCell API Parameters

Parameter	Type	Default Value
`a`	`List<Integer>`	—
`b`	`List<Integer>`	—
`content`	`GUIViewElement`	—